Name :- Aaryan N. Raorane Roll No :- 5004

Branch :- AI-DS batch : A1 SAP ID : 60018250070

Experiment No 5 : Numpy

Aim : To understand and apply basic and advanced Numpy operations and built in matrix functions, focusing on their application in data science and machine learning tasks.

Theory :

1. What is array slicing ? Provide an example where you extract a sub array from a 2D Numpy array.

=> Array slicing is a technique in Numpy that allows extraction of a contiguous portion of an array by specifying index ranges along each dimension. For a 2D array, the syntax is

```
array[row_start : row_end, col_start : col_end]
```

The indices follow Python conventions, where the start index is inclusive and the end index is exclusive. Slicing returns a view, not copy, meaning modifications to the slice affect the original array

Example :

```
import numpy as np
arr = np.array([[10, 20, 30, 40],
                [50, 60, 70, 80],
                [90, 100, 110, 120]])
sub = arr[0:2, 1:3]
print("original array :")
print(arr)
```

Sundaram FOR EDUCATIONAL USE

```
print ("Extracted subarray:")
print (sub)
```

output:

```
Original Array:
[[10 20 30 40]
 [50 60 70 80]
 [90 100 110 120]]
Extracted Subarray:
[[20 30]
 [60 70]]
```

2. Describe element-wise operations in Numpy. Why are they useful in data science applications?

=> Elementwise operations in Numpy are arithmetic or mathematical operations applied independently to each corresponding pair of elements across two arrays of the same shape. Numpy vectorizes these computations internally using optimized C routines, which eliminates the need for explicit Python loops and result in significantly faster execution. In data science, elementwise operations are widely used for feature scaling, computing residuals between predicted and actual value, applying transformations such as squaring or taking square roots across an entire dataset, and performing column wise arithmetic on numerical data.

Sundaram®
FOR EDUCATIONAL USE

Example :-

```
import numpy as np
a = np.array([2,4,6,8])
b = np.array([1,3,5,7])

print("a+b:", a+b)
print("a*b:", a*b)
print("a**2:", a**2)
print("np.sqrt(a):", np.sqrt(a))
```

output :-

```
a+b: [3 7 11 15]
a*b: [2 12 30 56]
a**2: [4 16 36 64]
np.sqrt(a): [1.41421356 2.
             2.44948974 2.82842712]
```

3. What is the purpose of using ~~like np~~ functions like np.zeros(), np.ones(), and np.empty()? when would you use each?

⇒ These three functions are used to create Numpy arrays of a specified shape with requiring explicit data input. Each serves a distinct initialization purpose.

i) np.zeros(shape) creates an array where every element is initialized to 0.0 It is used when a zero ~~beta~~ baseline is required, such as initializing weight matrices before training a neural network or creating placeholder arrays for accumulating sums.

Sundaram
FOR EDUCATIONAL USE

ii) np.ones(shape) creates an array where every element to 1.0. It is commonly used to construct bias vectors, mask arrays, or initial arrays where a multiplicative identity is needed.

iii) np.empty(shape) creates an array without initializing its values. The memory is allocated but the contents are arbitary (garbage values from previously stored data). It is the fastest of the three fn and is appropriate only when the array will be completely overwritten before use, making it useful in performance critical numerical routines.

Example: import numpy as np

```
z = np.zeros((2,3))
o = np.ones((2,3))
e = np.empty((2,3))
print("np.zeros((2,3)):\n", z)
print("np.ones((2,3)):\n", o)
print("np.empty((2,3)):\n", e)
```

Output:
```
np.zeros((2,3)):
[[0. 0. 0.]
 [0. 0. 0.]]
np.ones((2,3)):
[[1. 1. 1.]
 [1. 1. 1.]]
```

Sundaram
FOR EDUCATIONAL USE

np.empty((2,3)):

```
[[1.0094556 6e-315  0.00000000e+000
  0.00000000e+000]
 [0.00000000e+000  0.00000000e+000
  0.00000000e+000]]
```

4. Explain the difference between np.array() and np.linspace(). Provide a use-case for each function.

=>

Feature	np.range()	np.linspace()
control parameter	step size (interval between values)	Number of evenly spaced points.
Endpoint included	No (stop is exclusive)	Yes (stop is included by default)
output data type	Inferred from arguments (int or float)	Always float 64
Primary use case	Iterating over integer index ranges	Generating uniform sample points for plots or interpolation.
Predictability of Endpoint	Endpoint may be omitted due to floating point rounding	Endpoint is always guaranteed.

Sundaram
FOR EDUCATIONAL USE

Use case for np.arange(): Generating indices for iteration, such as creating a sequence of pixel values from 0 to 255 in step of 1.

Use case for np.linspace(): Generating uniformly distributed x values for plotting a mathematical function or creating equally spaced sample points for signal processing.

Example: import numpy as np

```
r = np.arange(0,10,2)
print("np.arange(0,10,2):", r)

l = np.linspace(0,1,5)
print("np.linspace(0,1,5):", l)
```

Output: np.arange(0,10,2): [0 2 4 6 8]
np.linspace(0,1,5): [0 0.25 0.5 0.75 1.]

5. How does reshaping an array work in Numpy? What will happen if the total number of elements does not match the new shape?

=> Reshaping in Numpy refers to changing the dimensional structure of an array without altering any of its underlying data values. The total number of elements must remain identical before and after the reshape operation. Numpy simply reinterprets the flat memory

Sundaram®
FOR EDUCATIONAL USE

layout of the array under new shape specification. If the product of the new shape dimension does not equal the total number of Element in the orignal array, Numpy raises a ValueError at runtime. For example, an array of 12 elements can be reshaped to (3,4), (2, 6), (4,3), or (12,), but attempting to reshape it to (3, 5) which requires 15 elements will cause an Error

Example: import numpy as np

```
a = np.arange(1,13)
print("orignal:", a)

b = a.reshape(3, 4)
print("Reshaped to (3,4):\n", b)

try:
    c = a.reshape(3,5)
except ValueError as e:
    print("Error:", e)
```

output: orignal: [1 2 3 4 5 6 7 8 9 10 11 12]
Reshaped to (3, 4):
[[1 2 3 4]
[5 6 7 8]
[9 10 11 12]]
error: cannot reshape array of size 12 into shape (3,5)

Sundaram
FOR EDUCATIONAL USE

6. What is matrix multiplication in Numpy, and how is it different from element wise multiplication?

=>

Aspect	Elementwise Multiplication	Matrix multiplication
operator / function	* operator	@ operator or np.matmul()
operation performed	Multiplies each element at position (i,j) in A with the element at the same position (i,j) in B	computes the dot product of rows of A with columns of B
Shape requirement	Both arrays must have identical shapes	Number of columns in A must equal number of rows in B.
Result shape	Same as input arrays	depends on outer dimensions of operands
Mathematical meaning	No standard linear algebra equivalent	standard matrix product from linear algebra

Sundaram
FOR EDUCATIONAL USE

Typical use case	masking, scaling, Hadamard product in ML	Linear transformation, neural network forward pass, solving systems of equations.

Example: import numpy as np

```
A = np.array([[1,2], [3,4]])
B = np.array([[5,6], [7,8]])

print("Elementwise (A * B):\n", A * B)
print("Matrix Multiplication (A @ B):\n", A @ B)
```

Output:
```
Elementwise (A * B):
[[5 12]
 [21 32]]
Matrix Multiplication (A @ B):
[[19 22]
 [43 50]]
```

7. Explain what broadcasting is in Numpy. Provide an example where broadcasting simplifies operations on arrays of different shapes.

⟹ Broadcasting is the mechanism in Numpy that allows arithmetic operations to be performed between arrays of different shapes without explicitly copying the data; Numpy automatically stretches the smaller

Sundaram
FOR EDUCATIONAL USE

array along dimensions where it has size 1, so that it matches the shape of the larger array during computation. The rules governing broadcasting are applied by comparing dimension from the trailing end. Two dimensions are compatible if they are equal or if one of them is 1. If neither condition holds, Numpy raises a ValueError. Broadcasting avoids the memory overhead of creating expanded copies and allows concise, readable code.

Example: adding a row vector of shape (3,) to a matrix of shape (3,3). The row vector is broadcast across all three rows of the matrix

```
import numpy as np

matrix = np.array([ [1,2,3],
                    [4,5,6],
                    [7,8,9] ])
row = np.array([10, 20, 30])
result = matrix + row
print("Matrix + Row (Broadcasting):\n", result)
```

Output:
```
Matrix + Row (Broadcasting):
[[11 22 33]
 [14 25 36]
 [17 28 39]]
```

Sundaram
FOR EDUCATIONAL USE

8. Describe how you would normalize a dataset using Numpy. Why is normalization important in machine learning tasks?

=> Normalization scales all values in a dataset to fixed range, typically [0, 1], using the min-max formula:

$$normalized = (x - x_{min}) / (x_{max} - x_{min})$$

This transformation ensures that smallest value in the dataset maps to 0 and the largest maps to 1, with all other values scaled proportionally in between. Normalization is critical in machine learning for several reasons. Algorithms based on distance computation such as K-Nearest neighbours and support vector machines are heavily biased toward features with larger numerical scales if normalization is not applied. Gradient descent based algorithm converge significantly faster when all input features are on a comparable scale, avoiding situations where the loss surface is elongated along certain dimensions.

Example:
```
import numpy as np
data = np.array([10.0, 20.0, 30.0, 40.0, 50.0])
normalized = (data - data.min()) / (data.max() - data.min())
print("Original data:", data)
print("Normalized data:", normalized)
```

Sundaram
FOR EDUCATIONAL USE

output:

Original data: [10. 20. 30. 40. 50.]
Normalized data: [0. 0.25 0.5 0.75 1.]

9. What is a covariance Matrix, and how does numpy help in computing it? Explain its importance in data science.

⟹ A covariance Matrix is a square symmetric matrix where the entry at position (i, j) represents the covariance between the ith and jth features of a dataset. The diagonal entries represent the variance of each individual feature, while the off-diagonal entries indicate how two features vary together. A positive covariance between features i and j means they tend to increase and decrease together, while a negative covariance means they vary in opposite directions. Numpy computes the covariance matrix using np.cov(), where each row of the input matrix is treated as one variable (feature) and each column as one observation. The function applies the sample covariance formula, dividing by N minus 1 rather than N to produce an unbiased estimate. The covariance matrix is central to several data science techniques. In principal component analysis (PCA), the eigenvectors of the covariance

Sundaram
FOR EDUCATIONAL USE

matrix define the principal components, enabling dimensionality reduction in multivariate statistics, it is used for linear discriminant analysis, Mahalanobis distance computation, and gaussian distribution modelling

Example:

```
import numpy as np

data = np.array([[2,3,5],[7,4,6],[1,8,9]])
cov = np.cov(data)
print("Dataset:\n", data)
print("Covariance:\n", cov)
```

Output:

```
Dataset:
[[2 3 5]
 [7 4 6]
 [1 8 9]]
Covariance Matrix:
[[2.33333333 -0.33333333  5.5       ]
 [-0.33333333 2.33333333 -4.5       ]
 [5.5        -4.5         19.       ]]
```

Sundaram®
FOR EDUCATIONAL USE

conclusions:

This experiment showed that Numly offers an efficient framework for numerical computing, covering array creation, slicing, reshaping, and vectorized mathematical operations that eliminate explicit loops and boost performance. Matrix operations like multiplication and covariance computation confirmed Numly's accurate implementation of linear algebra fundamentals underlying ML algorithms such as PCA and linear regression. Techniques like min-max normalization and covariance computation, applied to real datasets, established clear link between Numly's API and core data science workflows.

Sundaram®
FOR EDUCATIONAL USE